Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 2, Lab 2

Serial and Parallel Programming with OpenMP

---

1 Ideas

- **Serial** — one worker (one core running the loop).
- **Parallel** — more than one worker at the same time, on the *same* machine.
- **OpenMP** — hints in C (`#pragma omp ...`), a small library, and `-fopenmp`. gcc creates threads. One computer, shared memory. Not MPI.
- `pthread_create` starts threads by hand. OpenMP starts them for a `for` loop with one hint.
- **Race** — `sum += i` is look, add, write. Two threads can both see 7 and both write 8. Total is usually too small. **Reduction** — `reduction(+:sum)`: private copies, then gcc adds them. **Coherence** is hardware, not this bug. Critical / lock is not used in these programs.

speedup $= T_{\text{serial}}/T_{\text{parallel}}$. Serial 8 s and parallel 2 s $\Rightarrow$ speedup $= 4$, not "saved 6 seconds".

2 Compile flag `-fopenmp`

The serial program has **no** `#pragma`, so it still runs on **one** thread. It still needs `-fopenmp` because it uses `omp_get_wtime()` (wall-clock: how long you waited).

`clock()` is not used here. `clock()` can add CPU time from every core, so four cores working 1 second can look like 4 seconds.

`omp_get_max_threads()` is how many OpenMP is *allowed* to hire. It can print 8 even in the serial program. The serial loop still uses **one** thread because there is no `#pragma`. `-fopenmp` links the timer; it does not turn a normal `for` into a team.

3 Commands

Terminal

```
nproc
nano serial.c
gcc -fopenmp serial.c -o serial
./serial

nano parallel.c
gcc -fopenmp parallel.c -o parallel
./parallel
export OMP_NUM_THREADS=2
./parallel
export OMP_NUM_THREADS=4
./parallel
```

`nproc` should be 2 or more, or parallel will not get faster.
nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot `./` → command not found.
Forgot `-fopenmp` → error on `omp.h`, or still one thread.

4 Serial program (one worker)

C

```
#include <stdio.h>
#include <omp.h>

#define N 100000000

int main(void) {
    long i;
    long sum = 0;
    double start, end;

    start = omp_get_wtime();
    for (i = 0; i < N; i++) {
        sum += i;
    }
    end = omp_get_wtime();

    printf("sum = %ld\n", sum);
    printf("serial time = %f seconds\n", end - start);
    return 0;
}
```

If the time is under 0.2 seconds, add a zero to `N` in both programs.

5 Parallel program (same loop)

The only new line:

C

```
#pragma omp parallel for reduction(+:sum)
```

C

```
#include <stdio.h>
#include <omp.h>

#define N 100000000

int main(void) {
    long i;
    long sum = 0;
    double start, end;

    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < N; i++) {
        sum += i;
    }
    end = omp_get_wtime();

    printf("sum = %ld\n", sum);
    printf("parallel time = %f seconds\n", end - start);
    printf("threads = %d\n", omp_get_max_threads());
    return 0;
}
```

The printed `sum` must match serial. If it does not, `reduction` is missing.

$$\text{speedup} = \frac{\text{serial time}}{\text{parallel time}}$$

Four threads is not always 4× (Amdahl). Use the times from this run.